Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

Objectif

Vous guider pour la création d'une application web Django fonctionnelle qui interagit avec des modèles de Machine Learning Logistique regression comme cas d'exemple.

❶ Module 1 : La Fondation - Préparer du projet

But

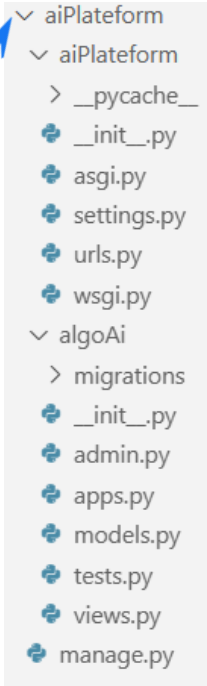
- ✓ Mettre en place un environnement de travail et création du projet

1. Installer les Librairies :
   - Dans la terminale, lancer la commande **pip install django**
   - django : Notre boîte à outils principale pour construire le site
2. Créer le Projet et l'Application :
   - La commande pour créer le projet nommée « aiPlateform » :

     django-admin startproject aiPlateform
   - Puis on accéde au dossier du projet via la commande

     cd aiPlateform
   - La commande pour créer l'application nommée « algoAi » :

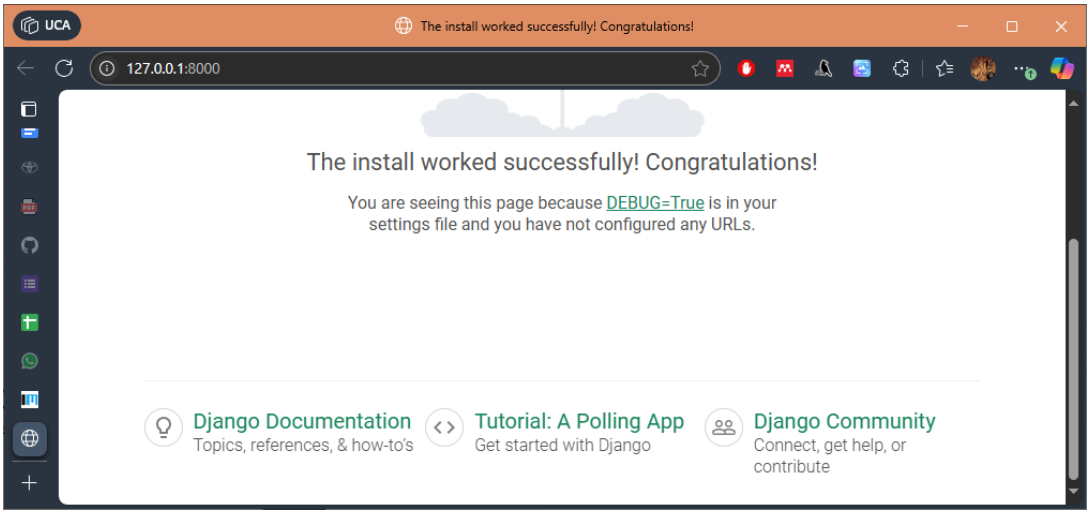
     python manage.py startapp algoAi



Analogie : « aiPlateform » est le terrain. « algoAi » est la première maison que vous construisez dessus.

- Pour tester si ça marche bien jusqu'au ici lancez la commande :

  python manage.py runserver



vous devez voir la page de bienvenue de Django avec la fusée ! Dans le navigateur

Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

❶ Module 1 : La Fondation - Préparer du projet (suite)

3. Enregistrer l'Application

- Ouvrir **settings.py** et ajouter **'algoAi'** dans **INSTALLED_APPS** comme dans la figure :

settings.py ×

aiPlateform > aiPlateform > settings.py

```
33  INSTALLED_APPS = [
34      'django.contrib.admin',
35      'django.contrib.auth',
36      'django.contrib.contenttypes',
37      'django.contrib.sessions',
38      'django.contrib.messages',
39      'django.contrib.staticfiles',
40      'algoAi'
41  ]
```

Analogie : Il faut déclarer officiellement la construction de maison (algoAi) à la mairie (aiPlateform) pour qu'elle soit reconnue.

❷ Module 2 : on commence le développement des interfaces

> **But**
> ✓ Créer la structure visible du site et les pages principales (sans la logique ML).

1. Le Standardiste (Le Fichier d'URLs principal) :

- Dans le fichier « **aiPlateform/urls.py** », ajouter une route qui transfère toutes les requêtes pour notre application vers « **algoAi** »

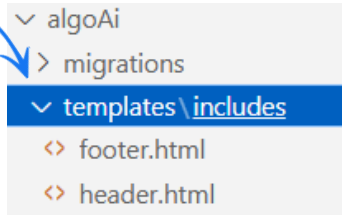
```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('algoAi.urls')),
]
```

Analogie : On installe un standardiste. Dès qu'un appel arrive pour le service « algoAi », il le redirige vers le bon bureau « ici c'est aiPlateform ».

2. Les Templates Réutilisables :

- Créer l'hierarchy de dossiers suivant « **algoAi/templates/includes/** » et les fichiers **header.html** et **footer.html** dans le dossier includes



Le contenu des fichiers `header.html` **et** `footer.html` **ici**

Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

❷ Module 2 : on commence le développement des interfaces (suite)

3. La Page d'Accueil (Première Vue et Template) :

- Action 1 (Le Cerveau) : Dans **views.py**, créer la fonction **index(request)**

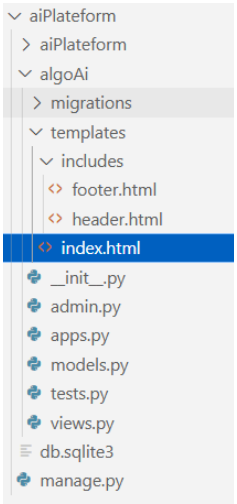
views.py

aiPlateform > algoAi > views.py

```python
from django.shortcuts import render

def index(request):
    return render(request, 'index.html')
```

- Action 2 (La Page) : Créer dans le dossier **« templates »** le fichier **« index.html »** et y inclure le *header* et le *footer*.

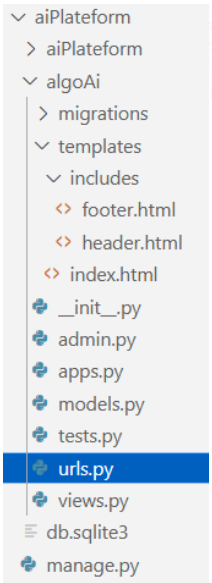


index.html

aiPlateform > algoAi > templates > index.html

```html
{% include "includes/header.html" %}
<div class="container">
    <h2> Page d'accueil </h2>
</div>
{% include "includes/footer.html" %}
```

- Action 3 (L'Adresse) : Dans « **algoAi** » ajouter un fichier nommé « **urls.py** », et créer dedans une route pour la fonction « **index** ».



urls.py

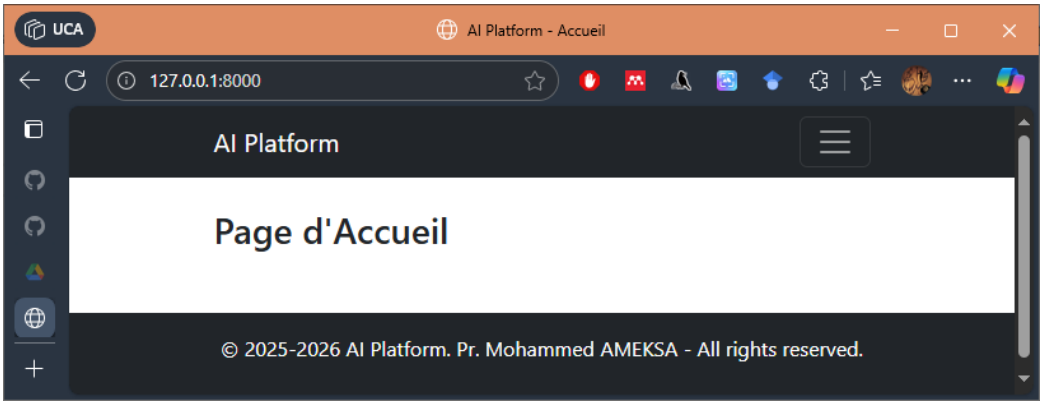
aiPlateform > algoAi > urls.py

```python
from django.urls import path
from . import views
urlpatterns = [
    path('', views.index, name='index'),
]
```

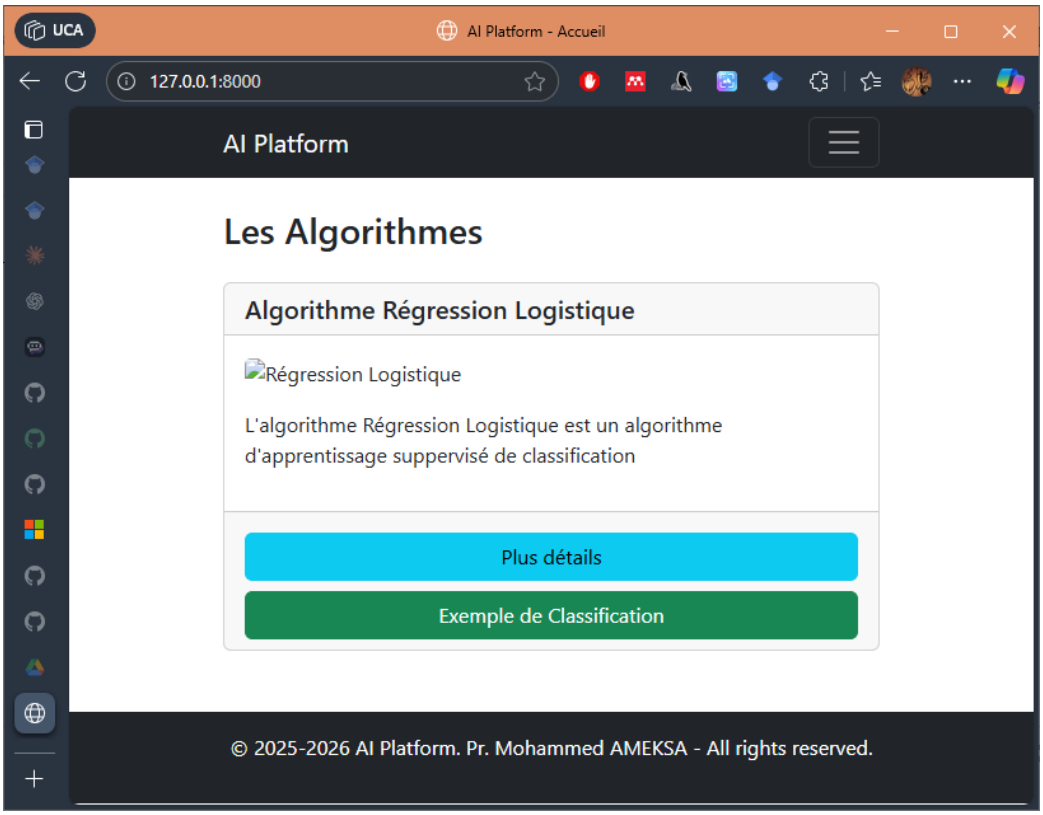
Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

❷ Module 2 : on commence le développement des interfaces (suite)

- Vérification : Lancer le serveur par la commade **« python manage.py runserver ».** La page d'accueil doit s'afficher



4. **Algorithme Régression Logistique** (#1 Carte) : Ajouter une carte bootstrap avec **2 boutons**:
   - Tâche 1 : Modifier le contenue du fichier **index.html**, en ajoutant une carte Bootstrap similaire à celle dans la figure suivante :

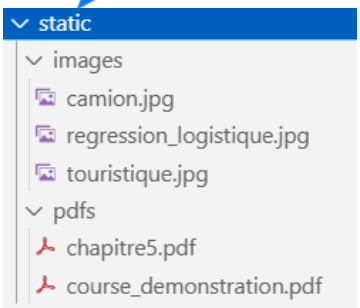


↳ Si vous n'arrive pas à le faire tout seule, veuillez voir le code dans le lien suivant **index.html**

Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

❷ Module 2 : on commence le développement des interfaces (suite)

4. **Algorithme Régression Logistique** (#1 Carte) : Ajouter une carte bootstrap avec **2 boutons**:
   - Tâche 2 : Comme vous voyez dans la figure précédente, l'image dans la carte ne s'affiche pas. La configuration des fichiers statiques (CSS, JS, images, pdfs, …) se fait en utilisant les étapes suivantes :
     - Ajouter un dossier nommé « **static** » dans le répertoire de l'application « **algoAi** »
       - Puis ajouter dedans l'ensemble des fichiers (images, pdfs, …) dont vous aurez besoin.
       - Pour notre cas nous auons besoin d'une image illustrant l'algorithme Régression Logistique « ex. regression_logistique.jpg»
       - Les deux fichiers pdfs (support de cours de l'algorithme Régression Logistique «chapitre5.pdf», et le support de l'atelier pratique – « course_demonstration.pdf»)
       - Bien évidement la même chose pour les autres images que nous aurons besoin « camion.jpg, et touristique.jpg»
       - En les organisant comme c'est figuré a coté



     - Maintenant modifier le code du fichier « **index.html** » comme suite
       - On ajouter **{% load static %}** au début de fichier « **index.html** »
       - Dans la balise **<img>** on change la valeur de l'attribut **src** par « **{% static 'images/regression_logistique.jpg' %}** »

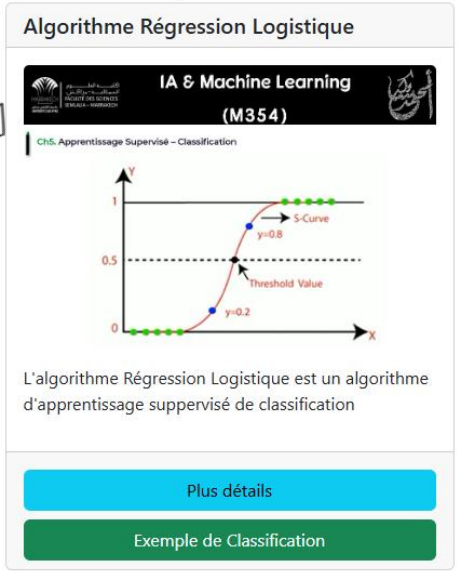
index.html ×

algoAi > templates > index.html

```
{% include "includes/header.html" %}
{% load static %}
<div class="container">

    <img src="{% static 'images/regression_logistique.jpg' %}" class="card-img-top">

</div>
{% include "includes/footer.html" %}
```



Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

❸ Module 3 : Configuration des deux boutons « plus détails » et « exemple de classification »

Plus détails | Exemple de Classification

But

✓ Rediriger l'utilisateur une fois clique sur l'un des deux boutons

- S'il clique sur plus détails on le redirige vers une page dont il retrouvera les détails sur l'algorithme Régression Logistique avec le support de cours sous format pdf
- Et pour le bouton « exemple de classification », on le redirige vers les détails de l'atelier pratique sur l'exemple de la base de données « véhicules classification » avec la possibilité de tester le modèle déjà entrainé dans l'atelier via la plateforme.

1. Le bouton « Plus détails » :

- Dans le fichier « **views.py** » on ajoute une 2ème fonction : **regLog_details(request)**

views.py

algoAi > views.py > ...

```python
from django.shortcuts import render
def regLog_details(request):
    return render(request, 'regLog_details.html')
```

- Maintenant, créer le fichier **regLog_details.html** dans le dossier « **templates** » comme suit :

regLog_details.html

algoAi > templates > regLog_details.html

```html
{% include "includes/header.html" %}
{% load static %}
<div class="container">
    <h2 class="mb-4">Algorithme de Régression Logistique</h2>
    <div class="row">
        <div class="col-md-6">
            <h3>Détails:</h3>
            <p>La régression logistique est un modèle de classification supervise  qui utilise une fonction logistique
            (ou sigmoïde) pour modéliser la probabilité qu'une observation appartienne à une classe particulière.</p>
        </div>
        <div class="col-md-6">
            <h3>Caractéristiques:</h3>
            <ul>
                <li>Sortie Probabiliste </li>
                <li>Seuil de Décision (0.5) </li>
                <li>Simplicité et Efficacité</li>
            </ul>
        </div>
    </div>
    <div>
        <iframe src="{% static 'pdfs/chapitre5.pdf' %}" width="100%" height="600px" style="border:1px solide #ccc;">
            Ce navigateur ne support pas PDFs. veuillez le télécharger :)
            <a href="{% static 'pdfs/chapitre5.pdf' %}"> Télécharger PDF</a>
        </iframe>
    </div>
</div>
{% include "includes/footer.html" %}
```

Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

❸ Module 3 : Configuration des deux boutons « plus détails » et « exemple de classification » (suite)

Plus détails | Exemple de Classification

- Associer un URL au view **regLog_details(request)** dans la liste des URLs :

urls.py

algoAi > urls.py > ...

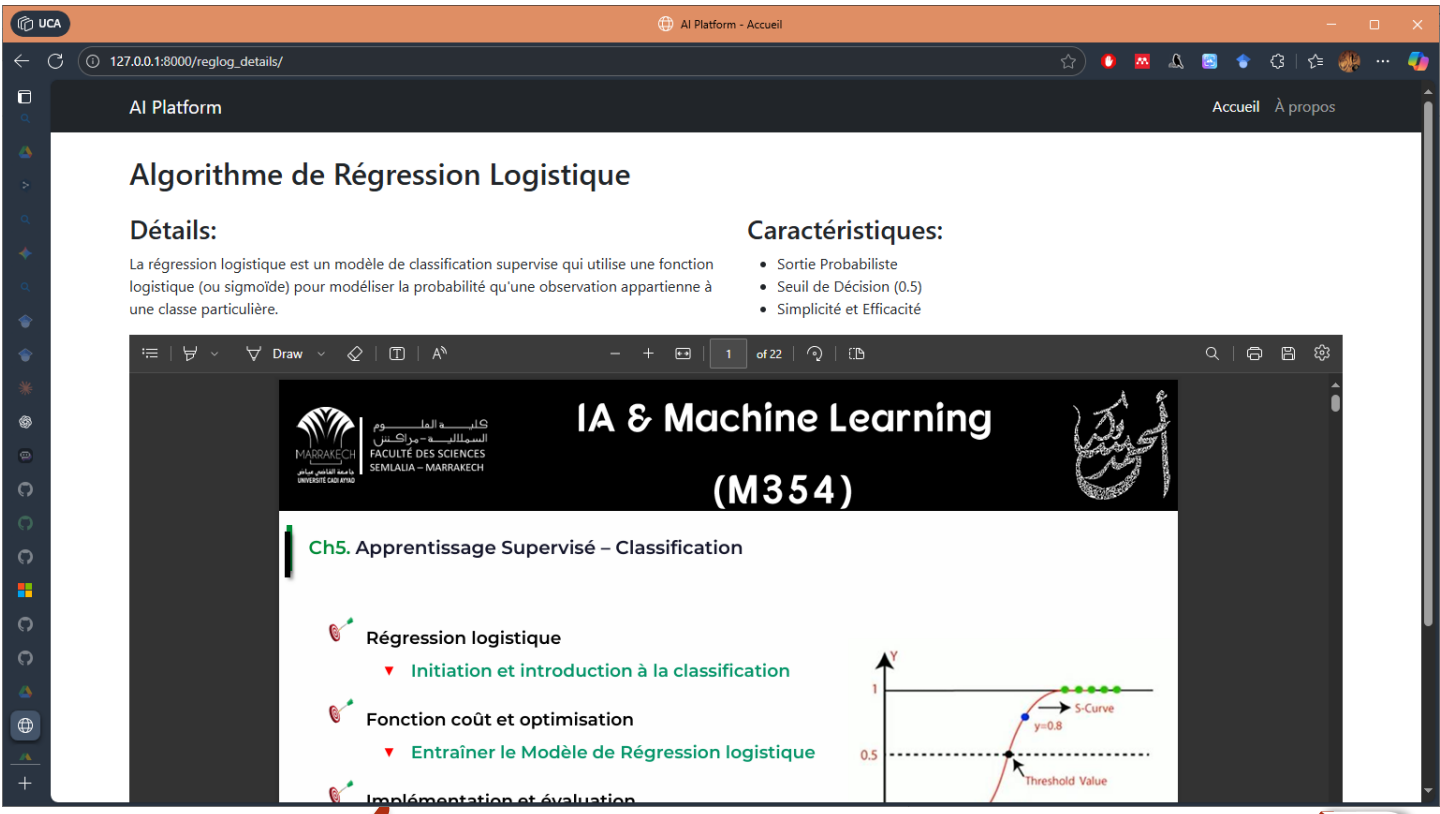
```python
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('reglog_details/', views.regLog_details)
]
```

- Configurer le buton « **plus détails** » dans le fichier « **index.html** » en changeant la valeur de l'attribut **href** comme suite :

```html
<a class="btn btn-info col-12 mt-2" href="{% url 'reglog_details' %}">Plus détails</a>
```

- Maintenant lancer l'application, puis clique sur le bouton « plus détails » pour vérifier



Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

❸ Module 3 : Configuration des deux boutons « plus détails » et « exemple de classification » (suite)

Plus détails | Exemple de Classification

2. Le bouton « Exemple de classification » :

- Dans le fichier « **views.py** » on ajoute une 3ème fonction : **regLog_atelier(request)**

views.py ×

algoAi > views.py > ...

```python
def regLog_atelier(request):
    return render(request, 'regLog_atelier.html')
```

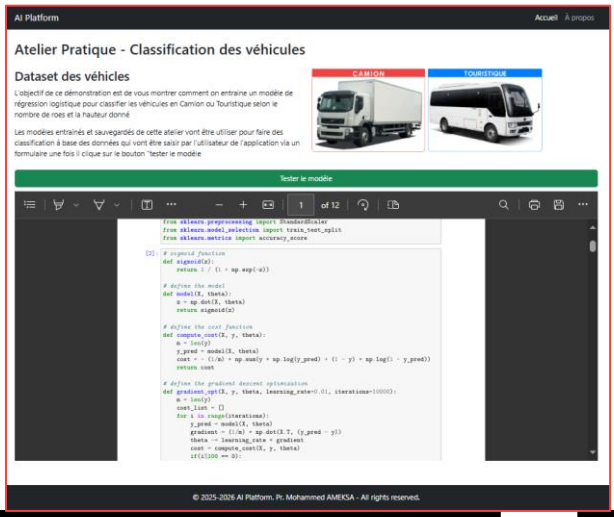
- Créer un fichier de template nommé « **regLog_atelier.html** » dans le dossier templates (adopter même structure que celle du fichier **regLog_details.html**) avec un bouton pour tester le modèle déjà entrainé

regLog_atelier.html ×

algoAi > templates > regLog_atelier.html

```html
{% include "includes/header.html" %}
{% load static %}
<div class="container">
    <h2 class="mb-4">Atelier Pratique - Classification des véhicules</h2>
    <div class="row">
        <div class="col-md-6">
            <h3>Dataset des véhicles</h3>
            <p>L'objectif de ce démonstration est de vous montrer comment on entraine un modèle de régression logistique pour classifier les véhicules en Camion ou Touristique selon le nombre de roes et la hauteur donné</p>
            <p>Les modèles entrainés et sauvegardés de cette atelier vont être utiliser pour faire des classification à base des données qui vont être saisir par l'utilisateur de l'application via un formulaire une fois il clique sur le bouton "tester le modèle</p>
        </div>
        <div class="col-md-6">
            <img src="{% static 'images/camion.jpg' %}" width="40%">
            <img src="{% static 'images/touristique.jpg' %}" width="40%">
        </div>
    </div>
    <div>
        <a class="btn btn-success col-12 mt-2 mb-2" href="#">Tester le modèle</a>
    </div>
    <div>
        <iframe src="{% static 'pdfs/course_demonstration.pdf' %}" width="100%" height="600px" style="border:1px solide #ccc;">
            Ce navigateur ne support pas PDFs. veuillez le télécharger :)
            <a href="{% static 'pdfs/course_demonstration.pdf' %}"> Télécharger PDF</a>
        </iframe>
    </div>

</div>
{% include "includes/footer.html" %}
```



Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

❸ Module 3 : Configuration des deux boutons « plus détails » et « exemple de classification » (suite)

Plus détails Exemple de Classification

2. Le bouton « Exemple de classification » :

- Ajouter un URL pour le view **regLog_atelier()** dans la liste des URLs :

urls.py

algoAi > urls.py > ...

```python
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('reglog_details/', views.regLog_details, name='reglog_details'),
    path('reglog_atelier/', views.regLog_atelier, name='reglog_atelier')
]
```

- Dans la page **index.html** associer l'URL crée au bouton « **exemple de classification** »

```html
<a class="btn btn-success col-12 mt-2" href="{% url 'reglog_atelier' %}">Exemple de Classification</a>
```

- Ajouter un autre **view regLog_tester()** pour rendre un formulaire à l'utilisateur afin de lui donner la possibilité de saisir des entrées au modèle (Régression Logistique)

views.py

algoAi > views.py > ...

```python
def regLog_tester(request) :
    return render(request, 'vehicles_form.html')
```

- Maintenant Créer un formulaire html « **vehicles_form.html** » dans le dossier templates avec une image comme celle dans la figure 2 sliders (input de type range) et un bouton de soumission du formulaire comme suit.

Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

❸ Module 3 : Configuration des deux boutons « plus détails » et « exemple de classification » (suite)

Plus détails | Exemple de Classification

vehicles_form.html

algoAi > templates > vehicles_form.html

```html
{% include "includes/header.html" %}
{% load static %}
<div class="container">
    <h2 class="mb-4">Les caractéristique de la véhicle</h2>
    <div class="ps-2 card">
        <div class="card-title">
            <img src="{% static 'images/illustration.JPG' %}" width="100%">
        </div>
        <div class="card-body">
            <form method="post" action="#" class="ps-4 pe-4 pt-4 pb-4">
                {% csrf_token %}
                <div class="mb-4">
                    <label for="hauteur" class="form-label">Hauteur (m)</label>
                    <input type="range" class="form-range" min="2" max="5.5" step="0.1" id="hauteur" name="hauteur"
                    value="3" oninput="hauteurValue.value=this.value">
                    <span class="slider-value" id="hauteurValue">3.0</span>
                </div>
                <div class="mb-4">
                    <label for="Nombre_de_roues" class="form-label">Nombre de roues</label>
                    <input type="range" class="form-range" min="4" max="18" step="2" id="Nombre_de_roues"
                    name="Nombre_de_roues" value="4" oninput="Nombre_de_rouesValue.value=this.value">
                    <span class="slider-value" id="Nombre_de_rouesValue">4</span>
                </div>
                <button type="submit" class="btn btn-primary col-12">Predict</button>
            </form>
        </div>
    </div>
</div>
{% include "includes/footer.html" %}
```

- **{% csrf_token %}** : C'est un badge de sécurité secret que Django donne à chaque formulaire. Si un formulaire est soumis sans ce badge, Django le rejette, car il pourrait venir d'un imposteur.

Vous pouvez rendre le formulaire plus interactif en ajoutant du code JS comme suite :

```html
<script>
    document.addEventListener('DOMContentLoaded', function(){
        hauteurValue.textContent = hauteur.value;
        Nombre_de_rouesValue.textContent = Nombre_de_roues.value;

    });
    hauteur.oninput = () => hauteurValue.textContent = hauteur.value
    Nombre_de_roues.oninput = () => Nombre_de_rouesValue.textContent = Nombre_de_roues.value
</script>
```

NB : Le but, une fois l'utilisateur soumit le formulaire, les données vont être transmis au modèle de la régression logistique déjà entrainé via un view associé à l'url dans l'attribut action (**regLog_prediction**)

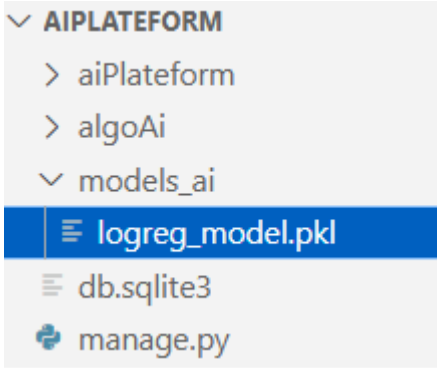
Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

4 Module 4 : Consommer le Modèle IA

But

✓ Connecter le formulaire HTML à notre modèle de prédiction

1. Configuration initiale
   - Créer le dossier « **models_ai** » dans le dossier de base du projet « **aiPlatform** » et y placer le fichier « **logreg_model.pkl** » le modèle déjà entrainé.



logreg_model.pkl est le modèle sauvegardé de l'atelier démonstration (classification des véhicules)

2. Création de view **regLog_prediction :** qui serve à recevoir les données du formulaire saisi par l'utilisateur puis les transmet au modèle et redirige l'utilisateur vers une page des résultats. Les taches de cet view on peut les structurer selon 6 tâches principales :
   - Tâche 1 : Recevoir le Colis ➔ On vérifie qu'on a bien reçu des données à traiter
   - Tâche 2 : Déballer le Colis ➔ On extrait les mesures de véhicule une par une
   - Tâche 3 : Réveiller l'Expert ➔ On charge notre modèle en mémoire
   - Tâche 4 : Poser la Question à l'Expert ➔ On lui donne les mesures et on attend sa réponse (0, ou 1)
   - Tâche 5 : Traduire la Réponse ➔ On traduit la réponse technique (un chiffre) en quelque chose de compréhensible (Camion/Touristique)
   - Tâche 6 : Préparer le Plateau-Repas (context) ➔ On met toutes les informations (le type de la véhicule, l'image correspondante, les données initiales) dans un « paquet » data à envoyer à la page de résultats

Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

4 Module 4 : Consommer le Modèle IA (suite)

2. Création de view **regLog_prediction :**

views.py

algoAi > views.py > ...

```python
def regLog_prediction(request):
    #Tâche 1: Recevoir le Colis
    if request.method == 'POST':
        # Tâche 2 : Déballer le Colis
        hauteur = float(request.POST.get('hauteur'))
        nbr_roues = float(request.POST.get('Nombre_de_roues'))
        #Tâche 3 : Réveiller l'Expert
        # cette fonction (load_models) est défini avant
        model = load_models('logreg_model.pkl')
        #Tâche 4 : Poser la Question à l'Expert
        prediction = model.predict([[hauteur,nbr_roues]])
        predicted_class = prediction[0]
        #Tâche 5 : Traduire la Réponse
        type_vehicules = {0: 'Camion', 1:'Touristique'}
        img_url = {'Camion':'images/camion.jpg', 'Touristique':'images/touristique.jpg'}
        pred_vehicule = type_vehicules[predicted_class]
        pred_img = img_url[pred_vehicule]
        #Tâche 6 : Préparer le Plateau-Repas (context)
        input_data = {
            'hauteur':hauteur,
            'nbr_roues':nbr_roues
        }
        context = {
            'type_vehicule':pred_vehicule,
            'img_vehicule':pred_img,
            'inital_data' : input_data
        }
        return render(request, 'reglog_results.html',context)
    return render(request,'vehicles_form.html')
```

- La fonction **load_models() :**

```python
def load_models(name):
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    models_dir = os.path.join(base_dir,'models_ai')
    model_path = os.path.join(models_dir, name)
    ml_model = joblib.load(model_path)
    return ml_model
```

Atelier Pratique : Votre Premier Site Web de Machine Learning avec Django

4 Module 4 : Consommer le Modèle IA (suite)

- Créer les URLs pour les deux derniers views : **reglog_tester()** et **reglog_prediction()**

urls.py

algoAi > urls.py > ...

```python
urlpatterns = [
    path('', views.index, name='index'),
    path('reglog_details/', views.regLog_details, name='reglog_details'),
    path('reglog_atelier/', views.regLog_atelier, name='reglog_atelier'),
    path('reglog_tester/', views.regLog_tester, name="reglog_tester"),
    path('reglog_prediction', views.regLog_prediction, name='reglog_prediction'),
]
```

- Associer l'URL « **reglog_tester** » au bouton « **Tester le modèle** » dans la page html « **reglog_atelier.html** »

```html
<a class="btn btn-success col-12 mt-2 mb-2" href="{% url 'reglog_tester' %}">Tester le modèle</a>
```

- Associer l'URL « **reglog_prediction** » à l'attribut « **action** » du formulaire dans la page html « **vehicules_form.html** »

```html
<form method="post" action="{% url 'reglog_prediction' %}" class="ps-4 pe-4 pt-4 pb-4">
```

3. Affichage des résultats de la prédiction

reglog_results.html

algoAi > templates > reglog_results.html

```html
{% include "includes/header.html" %}
{% load static %}
<div class="container">
    <div class="card mb-3">
        <div class="card-header">
            <strong>Résultat de classification</strong>
        </div>
        <div class="card-body">
            <div class="row">
                <div class="col-md-6">
                    <h5 class="card-title"> Type de véhicule prédit :
                        <span class="badge bg-primary">{{type_vehicule}}</span>
                    </h5>
                    <img src="{% static img_vehicule %}" class="img thumbnail card-img-top" style="width:350px">
                </div>
                <div class="col-md-6">
                    <p class="card-text">
                        Le type du véhicule prédit basé sur les caractéristiques suivantes : </p>
                    {% for key, value in inital_data.items%}
                        <div><strong>{{key}}:</strong>{{value}}</div>
                    {% endfor %}
                </div>
            </div>
        </div>
    </div>
</div>
{% include "includes/footer.html" %}
```